

Min 25

Min_25 筛

定义

从此种筛法的思想方法来说，其又被称为「Extended Eratosthenes Sieve」。

由于其由 [Min_25](#) 发明并最早开始使用，故称「Min_25 筛」。

性质

其可以在 $O(\sqrt{n})$ 或 $O(n^{1/3})$ 的时间复杂度下解决一类 **积性函数** 的前缀和问题。

要求：是一个关于 x 可以快速求值的完全积性函数之和（例如多项式）； $f(p)$ 可以快速求值。

记号

- 如无特别说明，本节中所有记为 p 的变量的取值集合均为全体质数。
- $\pi(x)$ 表示不超过 x 的质数个数。
- $\omega(x)$ ，即 x 为质数时其值为 1，否则为 0。
- p_k ：全体质数中第 k 小的质数（如： $p_1=2$ ）。特别地，令 $p_0=1$ 。
- $\min_p(x)$ ，即 x 的最小质因数。特别地， $\min_p(1)=1$ ，时，其值为 1。
- $\mu(x)$ 表示 x 的 Möbius 函数。
- $\varphi(x)$ 表示 x 的欧拉函数。

解释

观察 $\pi(x)$ 的定义，可以发现答案即为 $\sum_{p \leq x} 1$ 。

考虑如何求出 $\sum_{p \leq x} f(p)$ 。通过枚举每个 p 的最小质因子及其次数可以得到递推式：

最后一步推导基于这样一个事实：对于满足 $p \leq \sqrt{x}$ 的 p ，有 $x/p \geq \sqrt{x}$ ，故 $\sum_{p \leq \sqrt{x}} f(p) \sum_{p \nmid x/p} 1$ 。其边界值即为 $\sum_{p \leq \sqrt{x}} f(p) \pi(x/p)$ 。

假设现在已经求出了所有的 $\sum_{p \leq x} f(p)$ ，那么有两种方式可以求出所有的 $\sum_{p \leq x} f(p)$ ：

1. 直接按照递推式计算。
2. 从大到小枚举 x/p 转移，仅当 $x/p \geq \sqrt{x}$ 时转移增加值不为零，故按照递推式后缀和优化即可。

现在考虑如何计算 $\sum_{p \leq x} f(p)$ 。

观察求 $\sum_{p \leq x} f(p)$ 的过程，容易发现 x/p 有且仅有 $\sqrt{x}$ 处的点值是有用的。

一般情况下， $f(x)$ 是一个关于 x 的低次多项式，可以表示为 $\sum_{i=0}^d a_i x^i$ 。

那么对于每个 p ，其对 $\sum_{p \leq x} f(p)$ 的贡献即为 $\sum_{p \leq x} f(p)$ 。

分开考虑每个 p 的贡献，问题就转变为了：给定 x ，对所有的 $p \leq \sqrt{x}$ ，求 $\sum_{p \leq x} f(p)$ 。

Notice：是完全积性函数！

于是设，即埃筛第轮筛完后剩下的数的值之和。

对于一个合数，必定有。设为不大于的最大质数，则，即在埃筛进行轮之后剩下的均为质数。考虑的边界值，显然为。（还记得吗？特别约定了）

对于转移，考虑埃筛的过程，分开讨论每部分的贡献，有：

1. 对于的部分，值不变，即。
2. 对于的部分，被筛掉的数必有质因子，即。
3. 对于第二部分，由于，满足的会被额外减去。这部分应当加回来，即。

则有：

复杂度分析

对于的计算，其第一种方法的时间复杂度被证明为（见 zzt 集训队论文 2.3）；

对于第二种方法，其本质即为洲阁筛的第二部分，在洲阁论文中也有提及（6.5.4），其时间复杂度被证明为。

对于的计算，事实上，其实现与洲阁筛第一部分是相同的。

考虑对于每个，只有在枚举满足的转移时会对时间复杂度产生贡献，则时间复杂度可估计为：

对于空间复杂度，可以发现不论是还是，其均只在处取有效点值，共个，仅记录有效值即可将空间复杂度优化至。

首先，通过一次数论分块可以得到所有有效值，用一个大小为的数组记录。对于有效值，记为在中的下标，易得：对于所有有效值，。

然后分开考虑小于等于的有效值和大于的有效值：对于小于等于的有效值，用一个数组记录其，即；对于大于的有效值，用一个数组记录，由于过大所以借助记录，即。

这样，就可以使用两个大小为的数组记录所有有效值的并查询。在计算或时，使用有效值的代替有效值作为下标，即可将空间复杂度优化至。

过程

对于的计算，我们实现时一般选择实现难度较低的第一种方法，其在数据规模较小时往往比第二种方法的表现要好；

对于的计算，直接按递推式实现即可。

对于，可以用线性筛预处理出来替代递推式中的。

相应地，递推式中的也可以用此方法预处理。

用 Extended Eratosthenes Sieve 求积性函数的前缀和时，应当明确以下几点：

- 如何快速（一般是线性时间复杂度）筛出前个值；
- 的多项式表示；
- 如何快速求出。

明确上述几点之后按顺序实现以下几部分即可：

1. 筛出 内的质数与前 个 值；
2. 对 多项式表示中的每一项筛出对应的 ，合并得到 的所有 个有用点值；
3. 按照 的递推式实现递归，求出 。

例题

求莫比乌斯函数的前缀和

求 。

易知 。

直接筛即可得到 的所有 个所需点值。

求欧拉函数的前缀和

求 。

首先易知 。

对于 的一次项，有 ；

对于 的常数项，有 。

筛两次加起来即可得到 的所有 个所需点值。

[「LOJ #6053」简单的函数](#)

给定：

易知 。

则按照筛 的方法筛，对 讨论一下即可。

此处给出一种 C++ 实现：

▼ 参考代码

```
1 /* 「LOJ #6053」简单的函数 */
2 #include <LTcmath>
3 #include <LTiostream>
4
5 constexpr int MAXS = 200000; // 2sqrt(n)
6 constexpr int mod = 1000000007;
7
8 template <typename x_t, typename y_t>
9 void inc(x_t &x, const y_t &y) {
10     x += y;
11     (mod <= x) && (x -= mod);
12 }
13
14 template <typename x_t, typename y_t>
15 void dec(x_t &x, const y_t &y) {
16     x -= y;
17     (x < 0) && (x += mod);
18 }
19
20 template <typename x_t, typename y_t>
21 int sum(const x_t &x, const y_t &y) {
22     return x + y < mod ? x + y : (x + y - mod);
23 }
24
25 template <typename x_t, typename y_t>
26 int sub(const x_t &x, const y_t &y) {
```

```

27 return x < y ? x - y + mod : (x - y);
28 }
29
30 template <typename _Tp>
31 int div2(const _Tp &x) {
32     return ((x & 1) ? x + mod : x) >> 1;
33 }
34
35 // 以上目的均为防负数和取模
36 template <typename _Tp>
37 long long sqrr11(const _Tp &x) { // 平方函数
38     return (long long)x * x;
39 }
40
41 int pri[MAXS / 7], lpf[MAXS + 1], spri[MAXS + 1], pcnt;
42
43 void sieve(const int &n) {
44     for (int i = 2; i <= n; ++i) {
45         if (lpf[i] == 0) { // 记录质数
46             lpf[i] = ++pcnt;
47             pri[lpf[i]] = i;
48             spri[pcnt] = sum(spri[pcnt - 1], i); // 前缀和
49         }
50         for (int j = 1, v; j <= lpf[i] && (v = i * pri[j]) <= n; ++j) lpf[v] = j;
51     }
52 }
53
54 long long global_n;
55 int lim;
56 int le[MAXS + 1], // x <= \sqrt{n}
57     ge[MAXS + 1]; // x > \sqrt{n}
58 #define idx(v) (v <= lim ? le[v] : ge[global_n / v])
59
60 int G[MAXS + 1][2], Fprime[MAXS + 1];
61 long long lis[MAXS + 1];
62 int cnt;
63
64 void init(const long long &n) {
65     for (long long i = 1, j, v; i <= n; i = n / j + 1) {
66         j = n / i;
67         v = j % mod;
68         lis[++cnt] = j;
69         (j <= lim ? le[j] : ge[global_n / j]) = cnt;
70         G[cnt][0] = sub(v, 111);
71         G[cnt][1] = div2((long long)(v + 211) * (v - 111) % mod);
72     }
73 }
74
75 void calcFprime() {
76     for (int k = 1; k <= pcnt; ++k) {
77         const int p = pri[k];
78         const long long sqrp = sqrr11(p);
79         for (int i = 1; lis[i] >= sqrp; ++i) {
80             const long long v = lis[i] / p;
81             const int id = idx(v);
82             dec(G[i][0], sub(G[id][0], k - 1));
83             dec(G[i][1], (long long)p * sub(G[id][1], spri[k - 1]) % mod);
84         }
85     }
86     /* F_prime = G_1 - G_0 */
87     for (int i = 1; i <= cnt; ++i) Fprime[i] = sub(G[i][1], G[i][0]);
88 }
89
90 int f_p(const int &p, const int &c) {
91     /* f(p^{c}) = p xor c */

```

```

92 return p ^ c;
93 }
94
95 int F(const int &k, const long long &n) {
96     if (n < pri[k] || n <= 1) return 0;
97     const int id = idx(n);
98     long long ans = Fprime[id] - (spri[k - 1] - (k - 1));
99     if (k == 1) ans += 2;
100    for (int i = k; i <= pcnt && sqrl1(pri[i]) <= n; ++i) {
101        long long pw = pri[i], pw2 = sqrl1(pw);
102        for (int c = 1; pw2 <= n; ++c, pw = pw2, pw2 *= pri[i])
103            ans +=
104                ((long long)f_p(pri[i], c) * F(i + 1, n / pw) + f_p(pri[i], c + 1)) %
105                mod;
106    }
107    return ans % mod;
108 }
109
110 using std::cin;
111 using std::cout;
112
113 int main() {
114     cin.tie(nullptr)->sync_with_stdio(false);
115     cin >> global_n;
116     lim = sqrt(global_n); // 上限
117
118     sieve(lim + 1000); // 预处理
119     init(global_n);
120     calcFprime();
121     cout << (F(1, global_n) + 111 + mod) % mod << '\n';
122
123     return 0;
124 }

```

本页面最近更新：2024/12/5 14:25:50, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [Marcythm](#), [CSPNOIP](#), [StudyingFather](#), [Xeonacid](#), [aofall](#), [Back1ght](#), [CoelacanthusHex](#), [Enter-tainer](#), [Great-designer](#), [Haohu Shen](#), [iamtwz](#), [lr1d](#), [kenlig](#), [Konano](#), [MegaOwler](#), [Persdre](#), [SamZhangQingChuan](#), [shuzhouliu](#), [Tiphereth-A](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用